

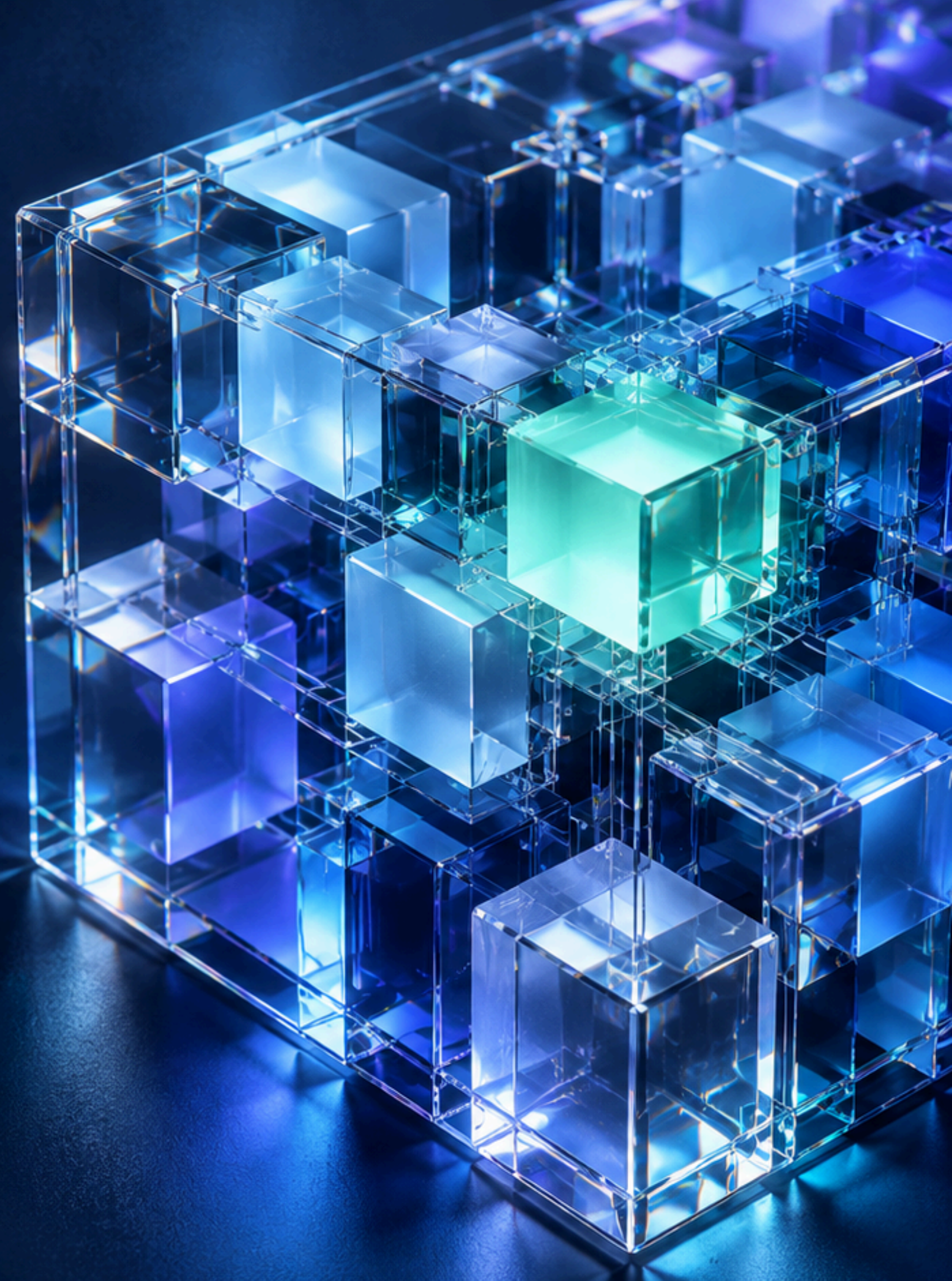
SOFTWARE QUALITY ASSURANCE

AI-Assisted Testing

เปรียบเทียบอัลกอริทึมสร้างกรณีทดสอบ
กับ Generative AI บน Defects4J

DynaMOSA / Reanimator / Gemini / Claude

CP353201 ภาคเรียนที่ 1/2569 มหาวิทยาลัยขอนแก่น
เสนอ ผศ.ดร.ชิตสุธา สุ่มเล็ก



สมาชิกภายในกลุ่ม



นายวงศ์ธร ธน.ยอด
673380425-2



นายอนันต์เอกก์ ใหญ่พงศกร
673380430-9



นายพัชรพล กองแก้ว
673380415-5

4 แนวทางในการทดลอง

อัลกอริทึมอัตโนมัติ 2 แบบ และโมเดล AI 2 แบบ

อัลกอริทึม / โมเดล	เครื่องมือ / รุ่น	แนวทาง
DynaMOSA	EvoSuite	Many-objective search
Reanimator	KEX 0.0.11 (concolic)	Backward search
Gemini	gemini-3.7-flash	AI เขียน JUnit 4 tests
Claude	claude-sonnet-5	AI เขียน JUnit 4 tests

ชุดข้อมูล Defects4J

Defects4J v3.0.1 เลือก classes.modified ของแต่ละบั๊ก

854

active bugs

17

โปรเจกต์ Java

1,067

Java targets สำหรับ AI

Frozen catalog 1,073 รายการ ตัด source ไม่พบ 3 และ non-Java 3 รายการ

DynaMOSA ผ่าน EvoSuite

Many-objective search

ค้นหาหลายเป้าหมาย

ปรับเป้าหมายตาม dependency ของสาขา
แล้วคัดเลือกและปรับชุดทดสอบด้วยการวิวัฒนาการ

300 วินาที/bug

Budget ของ algorithm DYNAMOSA

96.6%

Evaluation สำเร็จ 825/854 bugs (96.604%)



อัลกอริทึม Reanimator

ทำงานภายใน KEX 0.0.11 เพื่อสร้างโค้ดจาก symbolic input

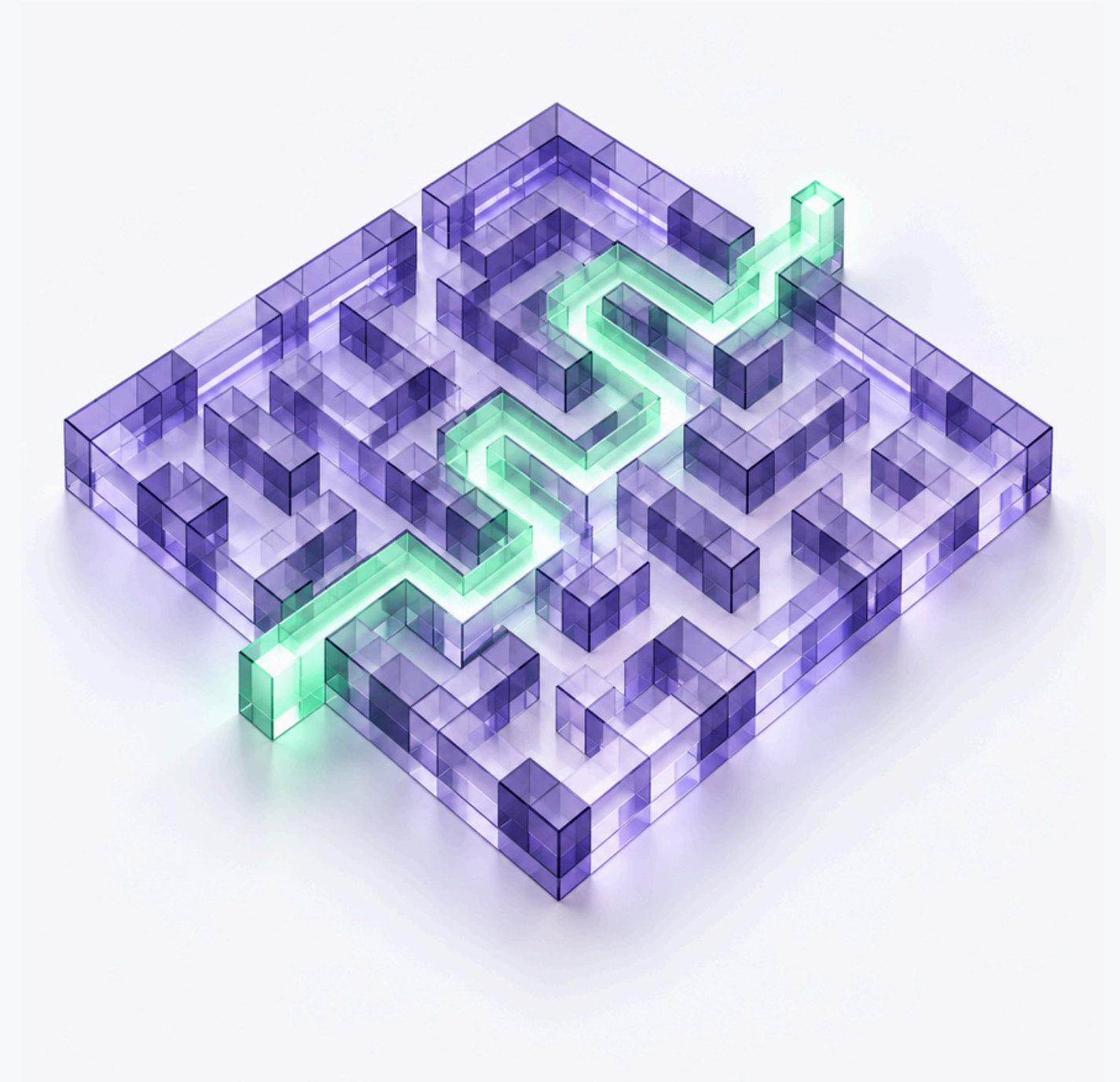
KEX: สืบหา Path ด้วย concolic execution

Reanimator: ใช้ backward search
สร้างลำดับเรียก public API ให้ได้ object state

65,099

candidate test classes จาก pipeline KEX

KEX 0.0.11 timeout 300 วินาที/target class
ข้อจำกัด: state explosion และแปลงบาง state ไม่ได้



การออกแบบ Prompt สำหรับ AI

Gemini ใช้ Markdown/plain structure Claude ใช้ XML tags

01 บริบท

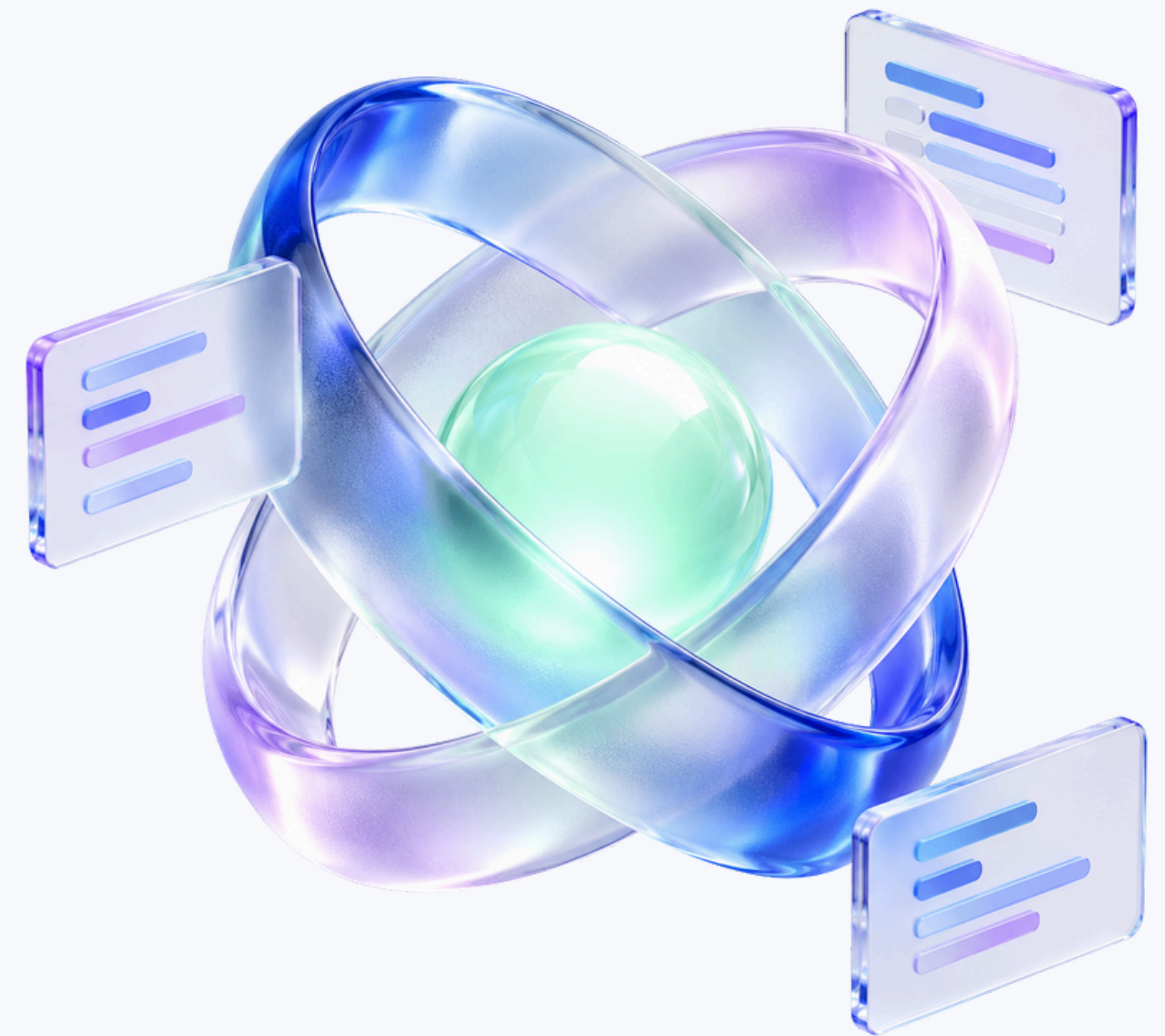
Package, class name และ target source
จาก frozen dataset

02 ข้อกำหนด

JUnit 4, public methods, normal/edge/exception
ห้าม mocking; Claude ระบุห้ามเดา API เพิ่ม

03 ผลลัพธ์

เฉพาะ Java code; ไม่มี prompt เฉพาะบั๊ก
Runner ไม่ส่ง dependency source เพิ่มเติม



วิธีการทดลอง

เป้าหมายเดียวกัน: classes.modified ของ buggy version

01

เตรียมข้อมูล

Algorithms: checkout / AI: frozen source

02

สร้าง Tests

DynaMOSA/EvoSuite,
Reanimator/KEX
Gemini, Claude

03

Compile และรัน

buggy/fixed: KEX ราย class
อีก 3 เทคนิค ราย test method

04

จำแนกผล

Candidate outcomes และ detected bugs

05

วัด Coverage

Line/Condition บน buggy

06

รวมผล

Master results 3,416 แถว

การตั้งค่าการทดลอง

1 รอบตาม protocol; AI runner มี retry และ resume

รายการ	ค่าที่ใช้
Environment	Ubuntu 22.04 (Docker), OpenJDK 11 default
DynaMOSA	300 วินาที/bug และ grace period 180 วินาที
KEX 0.0.11	concolic, timeout 300 วินาที/target class
AI	temperature 0.2, max_tokens 16,384, timeout 300 วินาที
JUnit / Coverage	15 วินาที/test/version และ 600 วินาที/bug

ติดตั้ง OpenJDK 8 และ 17 สำหรับ compatibility

ตัวชี้วัดการประเมิน

อัตราความสำเร็จและการตรวจพบบั๊กใช้คนละตัวหาร

854

bugs ในตัวหารรวม

Common Evaluation: 200 bugs

Common Coverage: 177 bugs

Evaluation Success Rate

bugs ที่ evaluate สำเร็จ ÷ 854

Overall / Conditional Fault Detection Rate

detected bugs ÷ 854 / evaluation success

Line / Condition Coverage

covered ÷ total เฉลี่ยเฉพาะ case สำเร็จ

Condition ใช้เฉพาะ total > 0

การจำแนกผลการทดสอบ

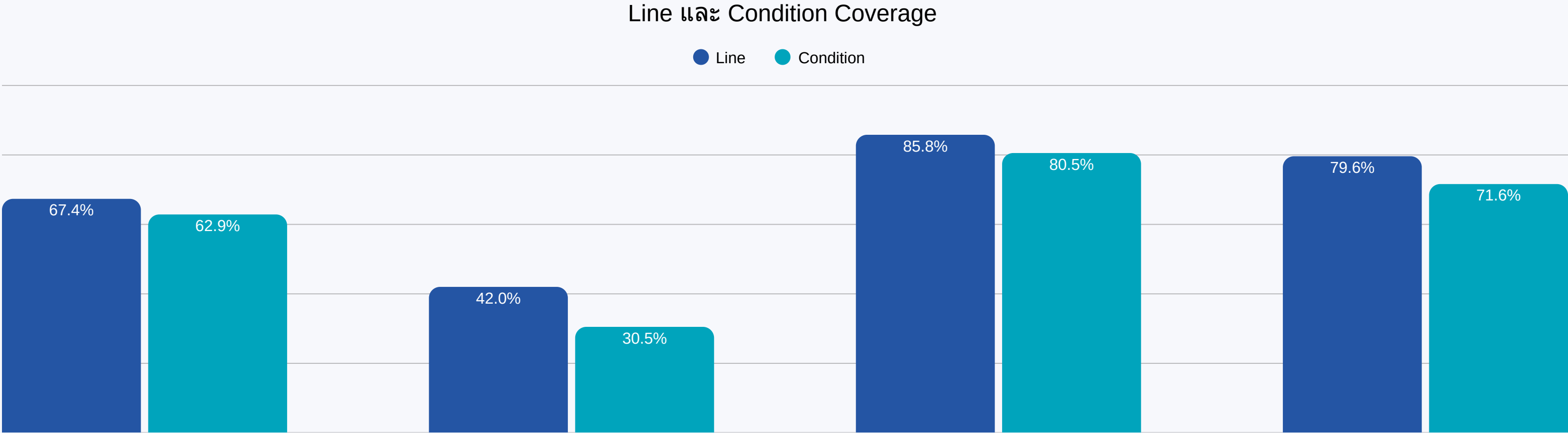
จำแนกระดับ candidate test แล้วจึงรวมเป็นผลรายบั๊ก

Buggy	Fixed	Classification
FAIL	PASS	bug_revealing
PASS	PASS	valid_non_revealing
FAIL	FAIL	invalid_or_unstable
PASS	FAIL	fixed_regression_or_unstable
TIMEOUT / อื่น ๆ	—	timeout_or_other

Detected bug ต้องมี bug_revealing อย่างน้อย 1 test

เปรียบเทียบ Coverage

ค่าเฉลี่ยเฉพาะกรณีที่วัดสำเร็จของแต่ละเทคนิค



n Line: 798 / 507 / 471 / 321 n Condition: 796 / 502 / 466 / 318

เรียง EvoSuite / KEX / Gemini / Claude • อ่าน Coverage Success Rate และ Common Set ประกอบ

สถานะ Evaluation ทั้งหมด

ระดับบั๊ก รวม 854 bugs ต่อเทคนิค

เทคนิค	Success	Compile failed	Missing
EvoSuite	825	8	21
KEX	684	0	170
Gemini	485	358	11
Claude	322	201	331

Success หมายถึงกระบวนการประเมินรันสำเร็จ แม้ผล test จะ FAIL

จุดเด่นและข้อจำกัด

ผลที่พบภายใต้ configuration ของ benchmark นี้

เทคนิค	จุดเด่น	ข้อจำกัด
EvoSuite	Eval success 96.604%	ตรวจพบ 2 bugs
KEX	Eval success 80.094%	Line 42.011%
Gemini	ตรวจพบ 56 bugs	Compile failed 358 bugs
Claude	Line 79.626%	Truncated 487 recorded calls

ข้อจำกัดและอุปสรรค

ขอบเขตที่ต้องพิจารณาเมื่อตีความผล

- 01** 1 รอบ/เทคนิค ยังไม่มีสถิติความแปรปรวนหลาย seed/generation
- 02** Candidate count: KEX นับ class; อีก 3 เทคนิคนับ method
- 03** AI มี compile errors และ output truncation; API success ไม่ยืนยัน compile
- 04** KEX Closure วัด Coverage สำเร็จ 2/174 bugs
- 05** KEX Lang-1 และ Math-1 แบบ diagnostic ไม่รวมใน Coverage หลัก
- 06** Docker ไม่ pin Defects4J commit; ต้องตรวจเวอร์ชันเมื่อทำซ้ำ
- 07** ผลขึ้นกับ model, prompt, budget, API และ environment

สรุปผลการทดลอง

ไม่มีเทคนิคใดดีที่สุดในทุกมิติ

56

บั๊กที่ Gemini ตรวจพบ

Overall FDR 6.557% Common FDR 10.5%
Mean Line Coverage 85.799% ในกลุ่มสำเร็จ

96.6%

Evaluation Success ของ EvoSuite

825/854 bugs (96.604%)
Coverage Success 93.443% สูงสุด

ผลสะท้อน benchmark และ configuration นี้ AI ยังต้องลด compile error และ output truncation

สิ่งที่ได้เรียนรู้และแนวทางการพัฒนา

ข้อเสนอสำหรับการทดลองในอนาคต

สิ่งที่ได้เรียนรู้

Coverage กับ Fault Detection เป็นคนละมิติ

จำนวน tests มากไม่รับประกันจำนวนบั๊กที่ตรวจพบ

Generation, Evaluation และ Coverage ต้อง
รายงานแยกกัน

แนวทางการพัฒนา

ปรับ dependency context และ assertions ของ
AI

แบ่ง output หรือใช้ continuation ลด truncation

เพิ่มหลาย seed/generation และทดลอง Hybrid

ขอบคุณครับ

Questions & Answers

github.com/Isharcmla/ProjectSQA

